

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Makkena Vineela(1BF24CS162)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING**

in

COMPUTER SCIENCE AND ENGINEERING



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Makkena Vineela(1BF24CS162)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	(a) Write program to obtain the Topological ordering of vertices in a given digraph. (b) LeetCode Program related to Topological sorting.	4-10
2	Implement Johnson Trotter algorithm to generate permutations.	11-13
3	(a) Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort. (b) LeetCode Program related to sorting.	14-20
4	(a) Sort a given set of N integer elements using Quick Sort technique and compute its time taken. (b) LeetCode Program related to sorting.	21-26
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	27-29
6	(a) Implement 0/1 Knapsack problem using dynamic programming. (5 Marks) (b) LeetCode Program related to Knapsack problem or Dynamic Programming.	30-34
7	(a) Implement All Pair Shortest paths problem using Floyd's algorithm. (b) LeetCode Program related to shortest distance calculation.	35-41
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	42-46
9	(a) Implement Fractional Knapsack using Greedy technique. (b) LeetCode Program related to Greedy Technique algorithms.	47-49
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	50-51
11	Implement "N-Queens Problem" using Backtracking.	52-54

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab Program - 1

1a) Write program to obtain the Topological ordering of vertices in a given digraph. (5 Marks)

code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int main() {
    int n, adj[MAX][MAX], indegree[MAX] = {0}, queue[MAX], front = 0, rear = 0;
    int topo[MAX], k = 0;

    printf("Enter number of vertices: ");
    if (scanf("%d", &n) != 1) return 1;

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (scanf("%d", &adj[i][j]) != 1) return 1;
            if (adj[i][j] == 1) {
                indegree[j]++;
            }
        }
    }

    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    while (front < rear) {
        int u = queue[front++];
        topo[k++] = u;

        for (int v = 0; v < n; v++) {
            if (adj[u][v] == 1) {
                indegree[v]--;
                if (indegree[v] == 0) {
```

```

        queue[rear++] = v;
    }
}
}

if (k != n) {
    printf("The graph contains a cycle. Topological ordering is not possible.\n");
} else {
    printf("Topological ordering: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", topo[i]);
    }
    printf("\n");
}

return 0;
}

```

Output:

```

"C:\Users\dell\Desktop\vinila" × + ▾
Enter number of vertices: 4
Enter adjacency matrix:
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0
Topological ordering: 0 1 2 3

Process returned 0 (0x0)   execution time : 156.789 s
Press any key to continue.

```

```
"C:\Users\dell\Desktop\vinila" × + ▾  
Enter number of vertices: 6  
Enter adjacency matrix:  
0 0 1 1 0 0  
0 0 1 0 1 0  
0 0 0 0 1 1  
0 0 0 0 0 1  
0 0 0 0 0 0  
0 0 0 0 0 0  
Topological ordering: 0 1 3 2 4 5  
  
Process returned 0 (0x0)   execution time : 65.480 s  
Press any key to continue.  
|
```

1b)LeetCode Program related to Topological sorting.

210. Course Schedule II

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: [0,1]

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].

Example 2:

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

Example 3:

Input: numCourses = 1, prerequisites = []

Output: [0]

Constraints:

1 <= numCourses <= 2000

0 <= prerequisites.length <= numCourses * (numCourses - 1)

prerequisites[i].length == 2

0 <= ai, bi < numCourses

ai != bi

All the pairs [ai, bi] are distinct.

Code:

```
#include <stdlib.h>
```

```
int* findOrder(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize, int* returnSize) {
    int* inDegree = (int*)calloc(numCourses, sizeof(int));
    int** adj = (int**)malloc(numCourses * sizeof(int*));
    int* adjSizes = (int*)calloc(numCourses, sizeof(int));
    int* adjCaps = (int*)calloc(numCourses, sizeof(int));

    for (int i = 0; i < numCourses; i++) {
        adjCaps[i] = 2;
        adj[i] = (int*)malloc(adjCaps[i] * sizeof(int));
    }

    for (int i = 0; i < prerequisitesSize; i++) {
        int dest = prerequisites[i][0];
        int src = prerequisites[i][1];
        inDegree[dest]++;

        if (adjSizes[src] == adjCaps[src]) {
            adjCaps[src] *= 2;
            adj[src] = (int*)realloc(adj[src], adjCaps[src] * sizeof(int));
        }
        adj[src][adjSizes[src]++] = dest;
    }

    int* queue = (int*)malloc(numCourses * sizeof(int));
    int head = 0, tail = 0;

    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) {
            queue[tail++] = i;
        }
    }

    int* result = (int*)malloc(numCourses * sizeof(int));
    int resultCount = 0;
```



```

while (head < tail) {
    int u = queue[head++];
    result[resultCount++] = u;

    for (int i = 0; i < adjSizes[u]; i++) {
        int v = adj[u][i];
        inDegree[v]--;
        if (inDegree[v] == 0) {
            queue[tail++] = v;
        }
    }
}

for (int i = 0; i < numCourses; i++) {
    free(adj[i]);
}
free(adj);
free(adjSizes);
free(adjCaps);
free(inDegree);
free(queue);

if (resultCount == numCourses) {
    *returnSize = numCourses;
    return result;
} else {
    free(result);
    *returnSize = 0;
    return NULL;
}
}

```

Output:

Testcase

Case 1

Case 2

Case 3

+

numCourses =
2

prerequisites =
[[1,0]]

</> Source ⓘ

Test Result

Accepted Runtime: 0 ms

Case 1

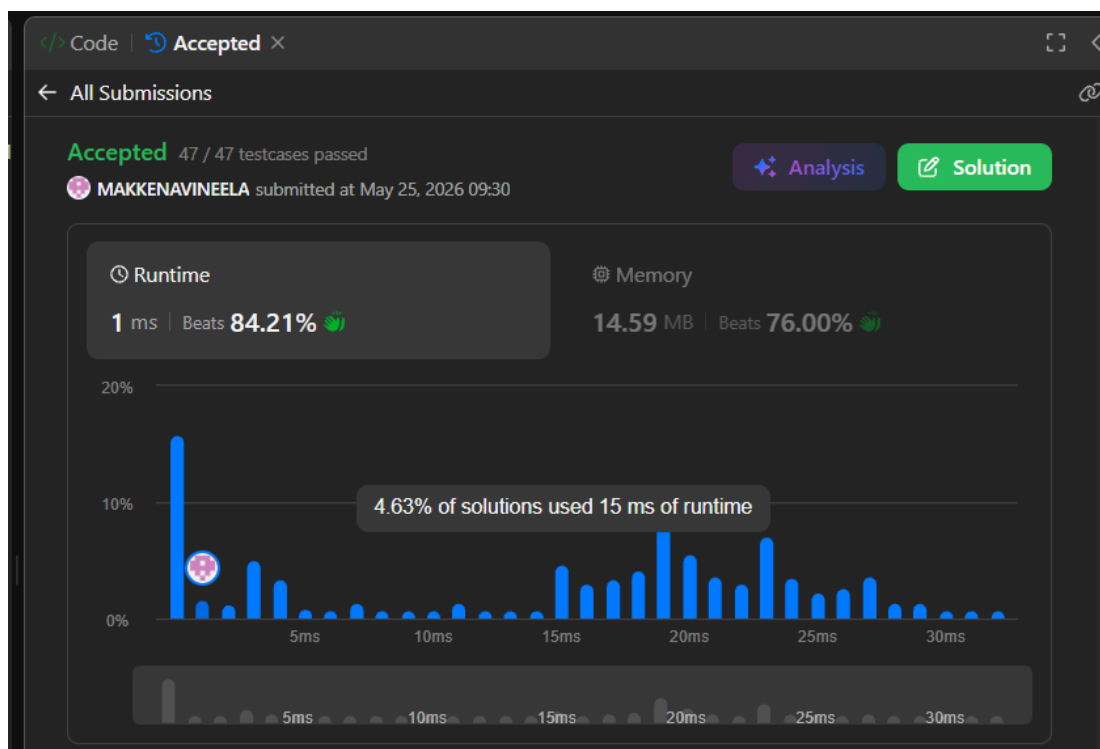
Case 2

Case 3

Input
numCourses =
2
prerequisites =
[[1,0]]

Output
[0,1]

Expected
[0,1]



Lab Program - 2

2) Implement Johnson Trotter algorithm to generate permutations.

code:

```
#include <stdio.h>
#include <stdbool.h>

#define LEFT false
#define RIGHT true

void printPermutation(int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

int getMobile(int a[], bool dir[], int n) {
    int mobile_prev = 0, mobile = 0;
    for (int i = 0; i < n; i++) {
        if (dir[i] == LEFT && i != 0) {
            if (a[i] > a[i - 1] && a[i] > mobile_prev) {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }
        if (dir[i] == RIGHT && i != n - 1) {
            if (a[i] > a[i + 1] && a[i] > mobile_prev) {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }
    }
    return mobile;
}

int findPos(int a[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (a[i] == x) return i;
    }
    return -1;
}
```

```

}

void swapInt(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

void swapBool(bool *x, bool *y) {
    bool temp = *x;
    *x = *y;
    *y = temp;
}

void johnsonTrotter(int n) {
    int a[n];
    bool dir[n];

    for (int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i] = LEFT;
    }

    printPermutation(a, n);

    while (1) {
        int mobile = getMobile(a, dir, n);
        if (mobile == 0) break;

        int pos = findPos(a, n, mobile);

        if (dir[pos] == LEFT) {
            swapInt(&a[pos], &a[pos - 1]);
            swapBool(&dir[pos], &dir[pos - 1]);
        } else if (dir[pos] == RIGHT) {
            swapInt(&a[pos], &a[pos + 1]);
            swapBool(&dir[pos], &dir[pos + 1]);
        }

        for (int i = 0; i < n; i++) {
            if (a[i] > mobile) {
                if (dir[i] == LEFT) dir[i] = RIGHT;
                else if (dir[i] == RIGHT) dir[i] = LEFT;
            }
        }
    }
}

```

```

    }

    printPermutation(a, n);
}
}

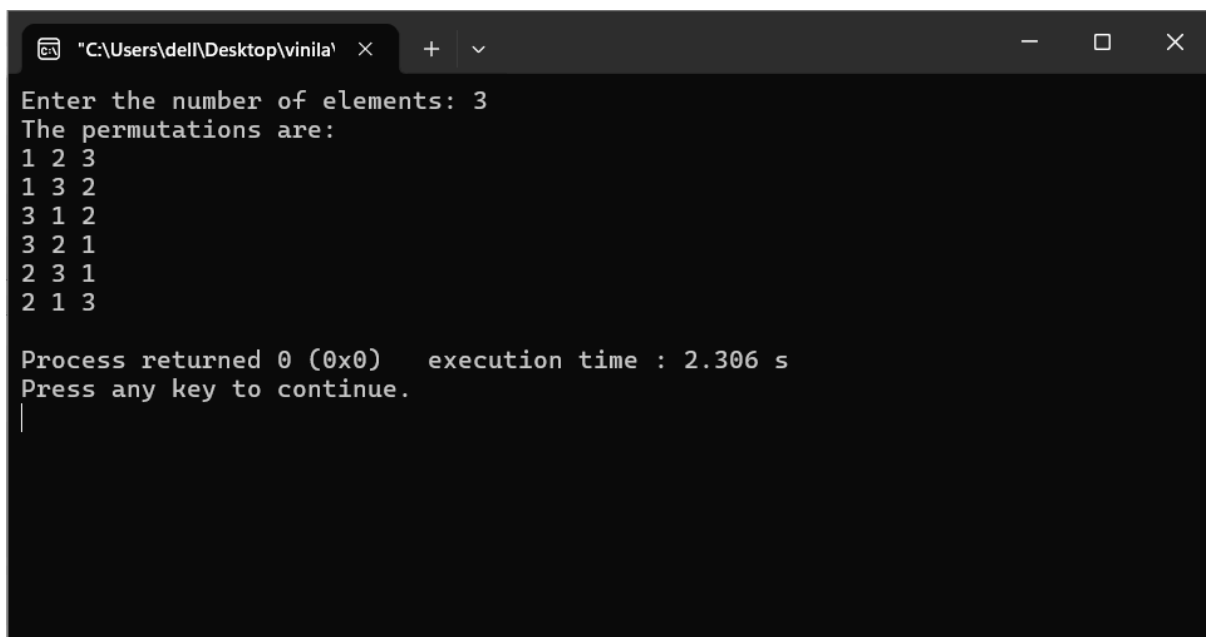
int main() {
    int n;
    printf("Enter the number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    printf("The permutations are:\n");
    johnsonTrotter(n);

    return 0;
}

```

Output:



```

"C:\Users\del\Desktop\vinila"
Enter the number of elements: 3
The permutations are:
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 2.306 s
Press any key to continue.
|

```

Lab Program - 3

3a) Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort. (5 Marks)

code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int low, int high, int mid);
void mergeSort(int a[], int low, int high);

void merge(int a[], int low, int high, int mid) {
    int i, j, k;
    int n1 = mid - low + 1;
    int n2 = high - mid;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++) {
        L[i] = a[low + i];
    }
    for (j = 0; j < n2; j++) {
        R[j] = a[mid + 1 + j];
    }

    i = 0;
    j = 0;
    k = low;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            a[k] = L[i];
            i++;
        } else {
            a[k] = R[j];
            j++;
        }
        k++;
    }
}
```

```

while (i < n1) {
    a[k] = L[i];
    i++;
    k++;
}
while (j < n2) {
    a[k] = R[j];
    j++;
    k++;
}
}

void mergeSort(int a[], int low, int high) {
    int mid;
    if (low < high) {
        mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, high, mid);
    }
}

int main() {
    int n;
    static int arr[1000000];
    clock_t start, end;
    double cpu_time;

    printf("Enter size: ");
    if (scanf("%d", &n) != 1) return 1;

    if (n > 100000) n = 100000;

    srand(time(NULL));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100000;
    }

    start = clock();
    mergeSort(arr, 0, n - 1);
    end = clock();

    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
}

```

```

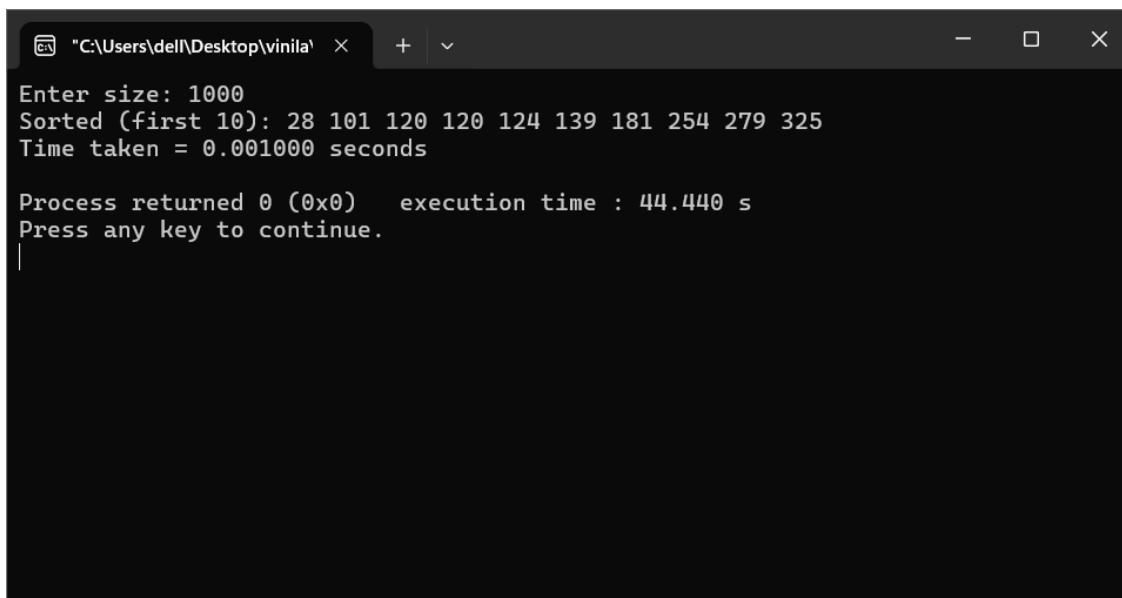
printf("Sorted (first 10): ");
for (int i = 0; i < 10 && i < n; i++) {
    printf("%d ", arr[i]);
}

printf("\nTime taken = %f seconds\n", cpu_time);

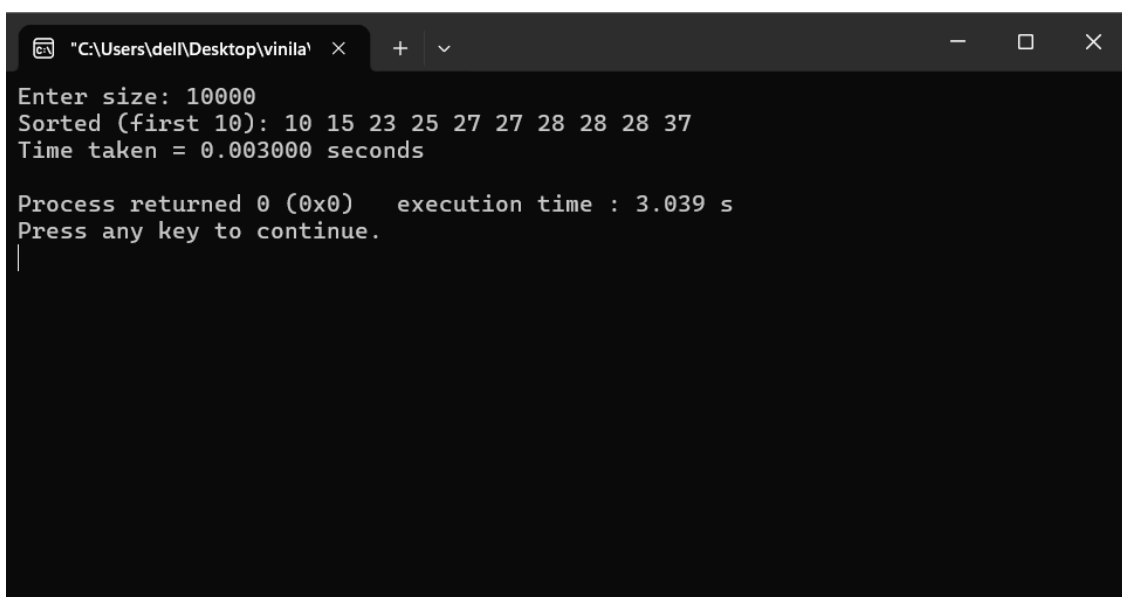
return 0;
}

```

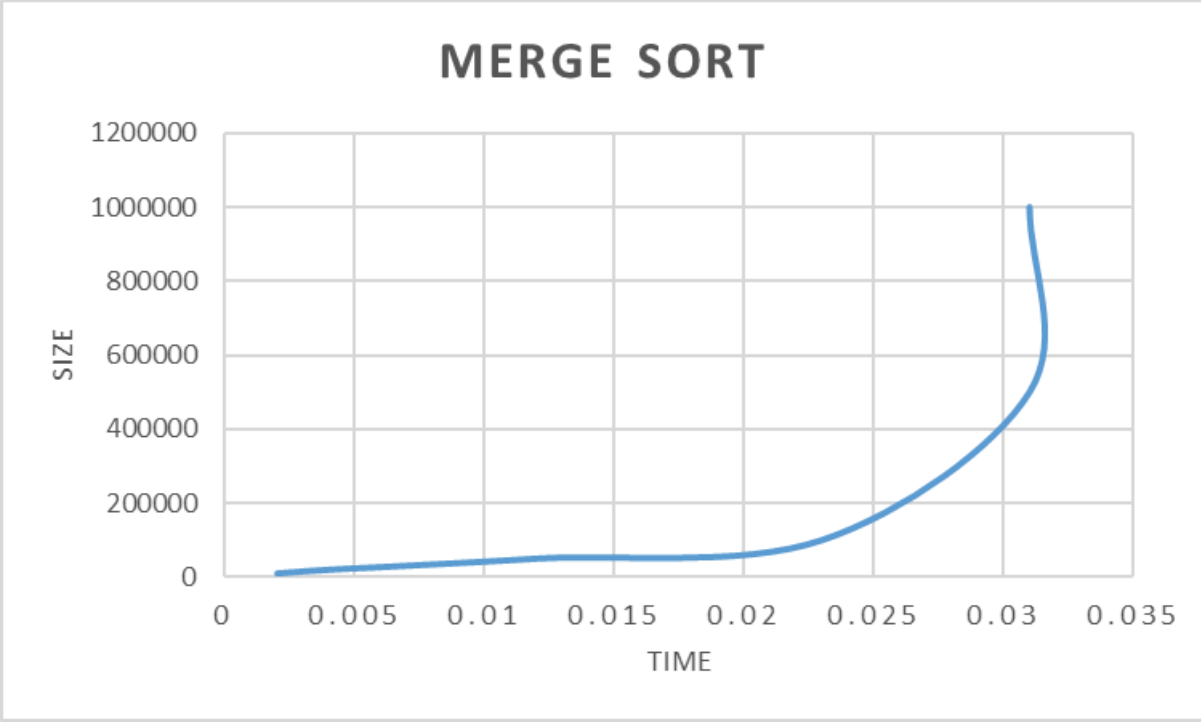
Output:



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\del\Desktop\vinila". The prompt displays the following text: "Enter size: 1000", "Sorted (first 10): 28 101 120 120 124 139 181 254 279 325", "Time taken = 0.001000 seconds", "Process returned 0 (0x0) execution time : 44.440 s", and "Press any key to continue." followed by a cursor on a new line.



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\del\Desktop\vinila". The prompt displays the following text: "Enter size: 10000", "Sorted (first 10): 10 15 23 25 27 27 28 28 28 37", "Time taken = 0.003000 seconds", "Process returned 0 (0x0) execution time : 3.039 s", and "Press any key to continue." followed by a cursor on a new line.



3b)LeetCode Program related to sorting.

912. Sort an Array

Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`

Output: `[1,2,3,5]`

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,1,1,2,0,0]`

Output: `[0,0,1,1,2,5]`

Explanation: Note that the values of `nums` are not necessarily unique.

Constraints:

$1 \leq \text{nums.length} \leq 5 \times 10^4$

$-5 \times 10^4 \leq \text{nums}[i] \leq 5 \times 10^4$

Code:

```
#include <stdio.h>
#include <stdlib.h>

void merge(int* nums, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* l = (int*)malloc(n1 * sizeof(int));
    int* r = (int*)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) {
        l[i] = nums[left + i];
    }
    for (int j = 0; j < n2; j++) {
```

```

        r[j] = nums[mid + 1 + j];
    }

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (l[i] <= r[j]) {
            nums[k] = l[i];
            i++;
        } else {
            nums[k] = r[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        nums[k] = l[i];
        i++;
        k++;
    }
    while (j < n2) {
        nums[k] = r[j];
        j++;
        k++;
    }

    free(l);
    free(r);
}

void mergesort(int* nums, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergesort(nums, left, mid);
    mergesort(nums, mid + 1, right);
    merge(nums, left, mid, right);
}

int* sortArray(int* nums, int numsSize, int* returnSize) {
    int left = 0;
    int right = numsSize - 1;

    mergesort(nums, left, right);

    *returnSize = numsSize;
    return nums;
}

```

Output:

☒ Testcase

Case 1

Case 2

+

nums =
[5,2,3,1]

</> Source ⓘ

> Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input
nums =
[5,2,3,1]

Output
[1,2,3,5]

Expected
[1,2,3,5]

♥ Contribute a testcase

</> Code | Accepted ×

← All Submissions

Accepted 21 / 21 testcases passed

MAKKENAVINEELA submitted at May 25, 2026 10:38

Analysis Solution

⌚ Runtime

344 ms | Beats 14.46%

💾 Memory

152.47 MB | Beats 20.96%

0.06% of solutions used 150 ms of runtime

2ms 50ms 97ms 145ms 192ms 240ms 288ms 335ms

Lab Program - 4

4) Sort a given set of N integer elements using Quick Sort technique and compute its time taken. (5 Marks)

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b);
int partition(int arr[], int low, int high);
void quciksort(int arr[], int low, int high);

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quciksort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quciksort(arr, low, pi - 1);
        quciksort(arr, pi + 1, high);
    }
}

int main() {
```

```

int sizes[] = {10000, 20000, 50000, 100000};
int num_test = sizeof(sizes) / sizeof(sizes[0]);

srand(time(NULL));

printf("%15s|%20s\n", "Elements (n)", "Time Taken (s)");
printf("-----\n");

for (int i = 0; i < num_test; i++) {
    int n = sizes[i];
    int *arr = (int*)malloc(n * sizeof(int));

    for (int j = 0; j < n; j++) {
        arr[j] = rand() % 100000;
    }

    clock_t start = clock();
    quicksort(arr, 0, n - 1);
    clock_t end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("%15d|%20f\n", n, time_taken);

    free(arr);
}

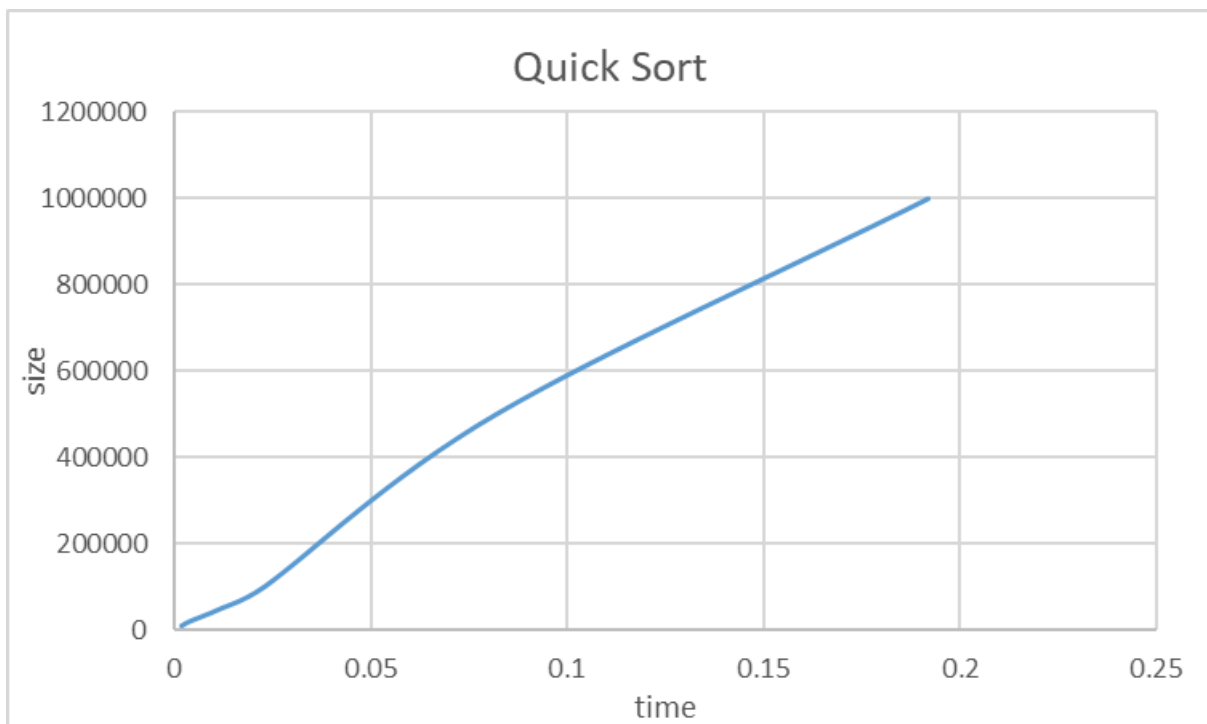
return 0;
}

```

Output:

```
"C:\Users\dell\Desktop\vinila" x + v
Elements (n)| Time Taken (s)
-----|-----
10000| 0.002000
20000| 0.004000
50000| 0.012000
100000| 0.023000
500000| 0.082000
1000000| 0.192000

Process returned 0 (0x0) execution time : 0.490 s
Press any key to continue.
```



4b) LeetCode Program related to sorting. (5 Marks)

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int findKthLargest(int* nums, int numsSize, int k) {
    srand(time(NULL));
    int low = 0;
    int high = numsSize - 1;
    int targetIdx = numsSize - k;

    while (low <= high) {
        int pivotIdx = low + rand() % (high - low + 1);
        int pivot = nums[pivotIdx];

        int lt = low;
        int gt = high;
        int i = low;

        while (i <= gt) {
            if (nums[i] < pivot) {
                swap(&nums[lt], &nums[i]);
                lt++;
                i++;
            } else if (nums[i] > pivot) {
                swap(&nums[i], &nums[gt]);
                gt--;
            } else {
                i++;
            }
        }

        if (targetIdx >= lt && targetIdx <= gt) {
            return nums[targetIdx];
        } else if (targetIdx < lt) {

```



```
        high = lt - 1;
    } else {
        low = gt + 1;
    }
}
return -1;
}
```

Output:

Testcase

Case 1 Case 2 +

nums =
[3,2,1,5,6,4]

k =
2

</> Source ?

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input
nums =
[3,2,1,5,6,4]
k =
2

Output
5

Expected
5

← All Submissions

Accepted 45 / 45 testcases passed

 **MAKKENAVINEELA** submitted at May 31, 2026 20:42

 Analysis

 Solution

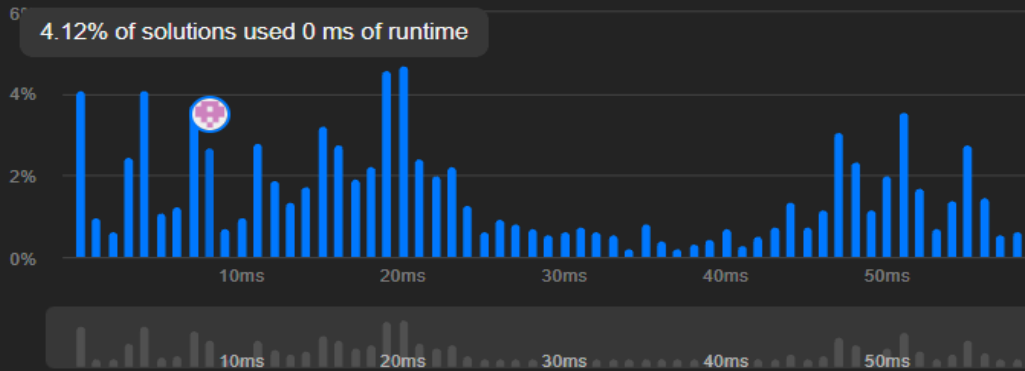
 Runtime

8 ms | Beats **81.53%** 

 Memory

14.73 MB | Beats **81.59%** 

4.12% of solutions used 0 ms of runtime



Lab Program - 5

5) Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i) {
    int larg = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[larg]) {
        larg = left;
    }
    if (right < n && arr[right] > arr[larg]) {
        larg = right;
    }
    if (larg != i) {
        swap(&arr[larg], &arr[i]);
        heapify(arr, n, larg);
    }
}

void heapsort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}
```

```

int main() {
    int arr[1000], n, i;
    clock_t start, end;
    double CPU_time;

    printf("Enter the size of the array (max 1000): ");
    if (scanf("%d", &n) != 1) return 1;

    if (n > 1000) n = 1000;

    srand(time(NULL));
    for (i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }
    printf("before sort:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    start = clock();
    heapsort(arr, n);
    end = clock();

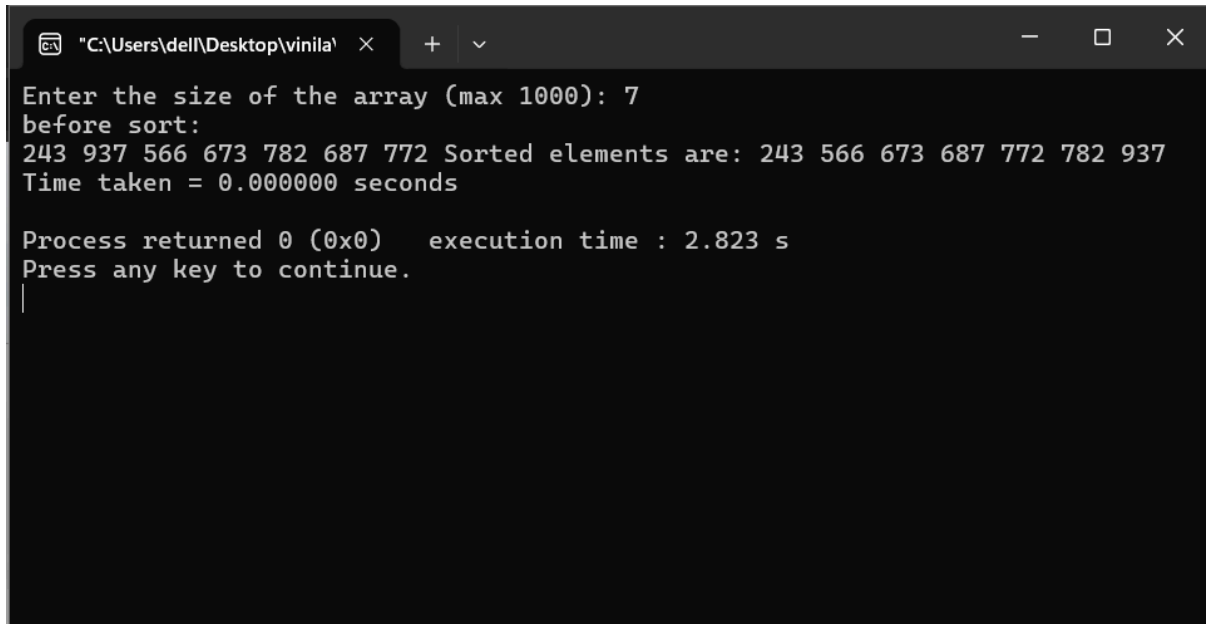
    CPU_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Sorted elements are: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    printf("\nTime taken = %f seconds\n", CPU_time);
    return 0;
}

```

Output:



```
"C:\Users\dell\Desktop\vinila" × + ▾
Enter the size of the array (max 1000): 7
before sort:
243 937 566 673 782 687 772 Sorted elements are: 243 566 673 687 772 782 937
Time taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 2.823 s
Press any key to continue.
|
```

Lab Program - 6

6a) Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)

code:

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int solveKnapsack(int profits[], int weights[], int n, int capacity) {
    if (capacity <= 0 || n == 0) {
        return 0;
    }

    int dp[n][capacity + 1];

    for (int i = 0; i < n; i++) {
        dp[i][0] = 0;
    }

    for (int c = 0; c <= capacity; c++) {
        if (weights[0] <= c) {
            dp[0][c] = profits[0];
        } else {
            dp[0][c] = 0;
        }
    }

    for (int i = 1; i < n; i++) {
        for (int c = 1; c <= capacity; c++) {
            int profit1 = 0;
            int profit2 = 0;

            if (weights[i] <= c) {
                profit1 = profits[i] + dp[i - 1][c - weights[i]];
            }
            profit2 = dp[i - 1][c];
        }
    }
}
```

```

        dp[i][c] = max(profit1, profit2);
    }
}

return dp[n - 1][capacity];
}

int main() {
    int n;
    printf("Enter number of items: ");
    scanf("%d", &n);

    int weights[n];
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &weights[i]);
    }

    int profits[n];
    printf("Enter profits of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &profits[i]);
    }

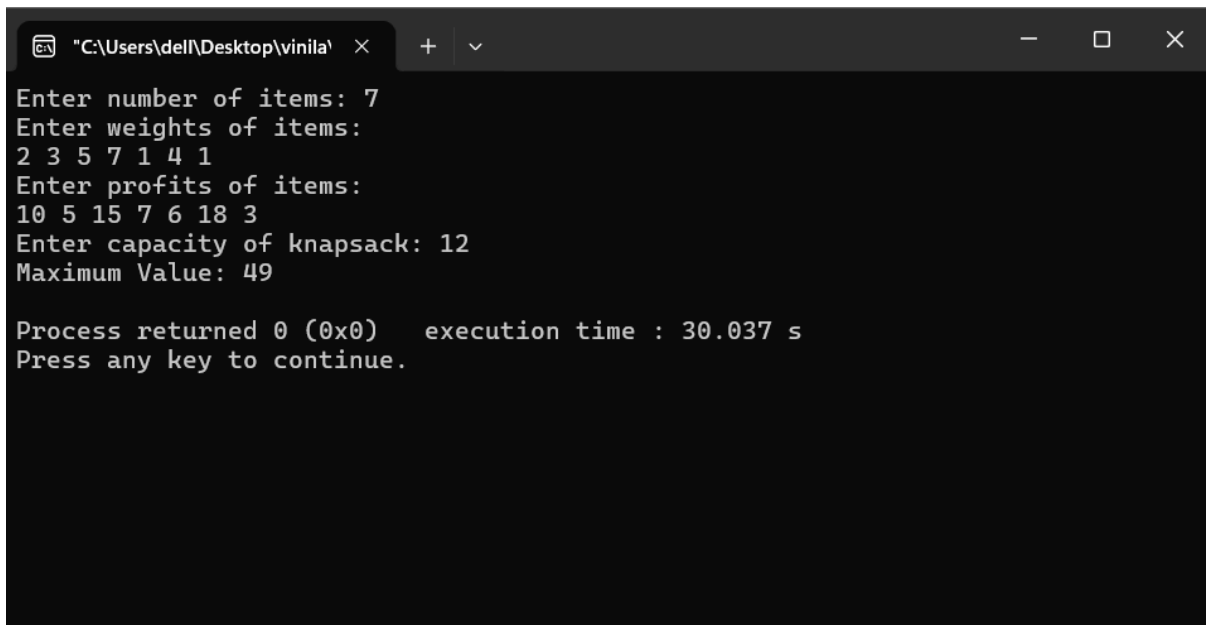
    int capacity;
    printf("Enter capacity of knapsack: ");
    scanf("%d", &capacity);

    int maxVal = solveKnapsack(profits, weights, n, capacity);
    printf("Maximum Value: %d\n", maxVal);

    return 0;
}

```

Output:



```
"C:\Users\dell\Desktop\vinila" x + v
Enter number of items: 7
Enter weights of items:
2 3 5 7 1 4 1
Enter profits of items:
10 5 15 7 6 18 3
Enter capacity of knapsack: 12
Maximum Value: 49

Process returned 0 (0x0)   execution time : 30.037 s
Press any key to continue.
```


6b) LeetCode Program related to Knapsack problem or Dynamic Programming.

416. Partition Equal Subset Sum

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `false` otherwise.

Example 1:

Input: `nums = [1,5,11,5]`

Output: `true`

Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`

Output: `false`

Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

`1 <= nums.length <= 200`

`1 <= nums[i] <= 100`

code:

```
#include <stdbool.h>
```

```
#include <string.h>
```

```
int memo[201][10001];
```

```
bool solve(int* nums, int n, int index, int target) {
```

```
    if (target == 0) return true;
```

```
    if (index >= n || target < 0) return false;
```

```
    if (memo[index][target] != -1) {  
        return memo[index][target];
```

```

    }
    bool included = false;
    if (nums[index] <= target) {
        included = solve(nums, n, index + 1, target - nums[index]);
    }

    bool excluded = solve(nums, n, index + 1, target);
    return memo[index][target] = (included || excluded);
}

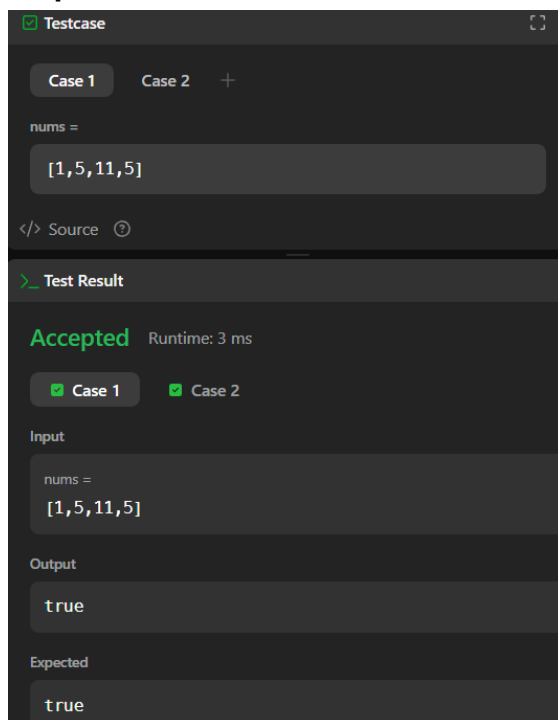
bool canPartition(int* nums, int numsSize) {
    int totalSum = 0;
    for (int i = 0; i < numsSize; i++) {
        totalSum += nums[i];
    }
    if (totalSum % 2 != 0) return false;

    int target = totalSum / 2;
    memset(memo, -1, sizeof(memo));

    return solve(nums, numsSize, 0, target);
}

```

output:



Lab Program - 7

7a) Implement All Pair Shortest paths problem using Floyd's algorithm. (5 Marks)

Code:

```
#include <stdio.h>
#define MAX 100
#define INF 99999

void floyd(int graph[MAX][MAX], int n) {
    int dist[MAX][MAX];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];
        }
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    printf("\nAll-Pairs Shortest Path Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
}
```

```

    }
}

int main() {
    int n, graph[MAX][MAX];

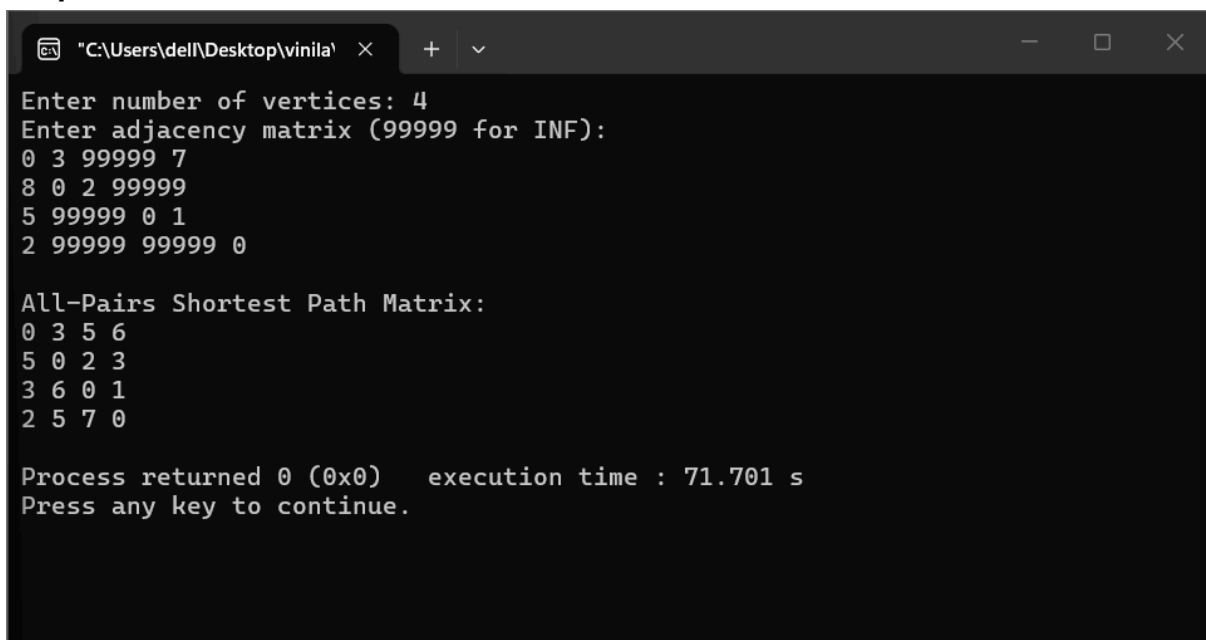
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (99999 for INF):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    floyd(graph, n);
    return 0;
}

```

Output:



```

"C:\Users\dell\Desktop\vinila"
Enter number of vertices: 4
Enter adjacency matrix (99999 for INF):
0 3 99999 7
8 0 2 99999
5 99999 0 1
2 99999 99999 0

All-Pairs Shortest Path Matrix:
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0

Process returned 0 (0x0)   execution time : 71.701 s
Press any key to continue.

```

7b)LeetCode Program related to shortest distance calculation.

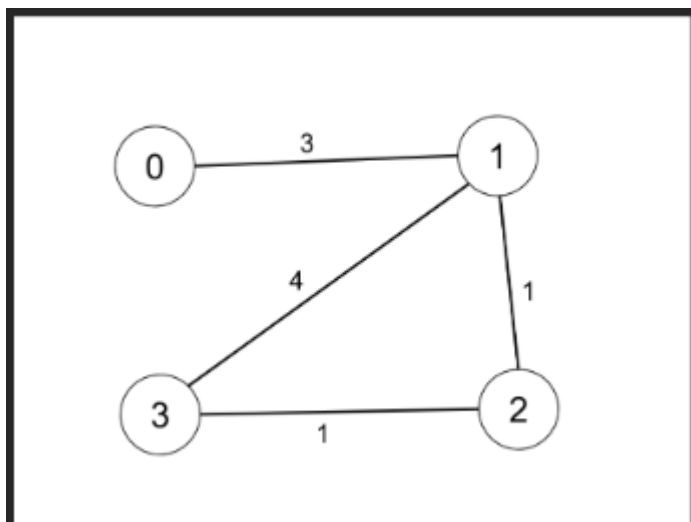
1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:



Input: $n = 4$, `edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`, `distanceThreshold = 4`

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 4` for each city are:

City 0 -> [City 1, City 2]

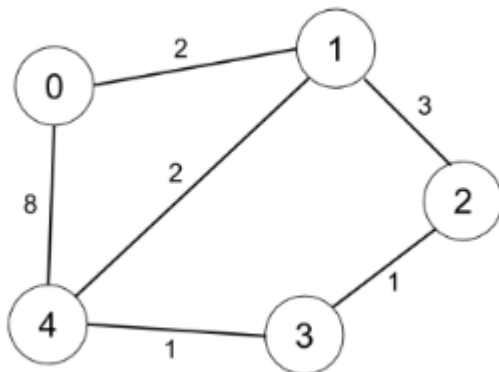
City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a `distanceThreshold = 4`, but we have to return city 3 since it has the greatest number.

Example 2:



Input: $n = 5$, $edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]$, $distanceThreshold = 2$

Output: 0

Explanation: The figure above describes the graph.

The neighboring cities at a $distanceThreshold = 2$ for each city are:

City 0 -> [City 1]

City 1 -> [City 0, City 4]

City 2 -> [City 3, City 4]

City 3 -> [City 2, City 4]

City 4 -> [City 1, City 2, City 3]

The city 0 has 1 neighboring city at a $distanceThreshold = 2$.

Constraints:

$2 \leq n \leq 100$

$1 \leq edges.length \leq n * (n - 1) / 2$

$edges[i].length == 3$

$0 \leq from_i < to_i < n$

$1 \leq weight_i, distanceThreshold \leq 10^4$

All pairs $(from_i, to_i)$ are distinct.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int
distanceThreshold) {
    int INF = 1e9;

    int** adj = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        adj[i] = (int*)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            if (i == j) {
                adj[i][j] = 0;
            } else {
                adj[i][j] = INF;
            }
        }
    }

    for (int i = 0; i < edgesSize; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int weight = edges[i][2];
        adj[u][v] = weight;
        adj[v][u] = weight;
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (adj[i][k] != INF && adj[k][j] != INF && adj[i][k] + adj[k][j] < adj[i][j]) {
                    adj[i][j] = adj[i][k] + adj[k][j];
                }
            }
        }
    }

    int node = 0;
    int minCount = n;

    for (int i = 0; i < n; i++) {
        int count = 0;
```

```

    for (int j = 0; j < n; j++) {
        if (i != j && adj[i][j] <= distanceThreshold) {
            count++;
        }
    }
    if (count <= minCount) {
        minCount = count;
        node = i;
    }
}

for (int i = 0; i < n; i++) {
    free(adj[i]);
}
free(adj);

return node;
}

```

output:

Testcase

Case 1

Case 2

+

n =

4

edges =

[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

distanceThreshold =

</> Source

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

n =

4

edges =

[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

distanceThreshold =

4

Accepted 54 / 54 testcases passed

MAKKENAVINEELA submitted at May 01, 2026 11:30

Analysis

Solution

✓ Approach ✓ Efficiency ✓ Code Style



Congratulations! You passed on your first attempt with this problem type. Your implementation of the all-pairs shortest path algorithm is spot on!

Approach

Current: Floyd-Warshall Algorithm / Dynamic Programming

Suggested: Floyd-Warshall Algorithm / Dynamic Programming

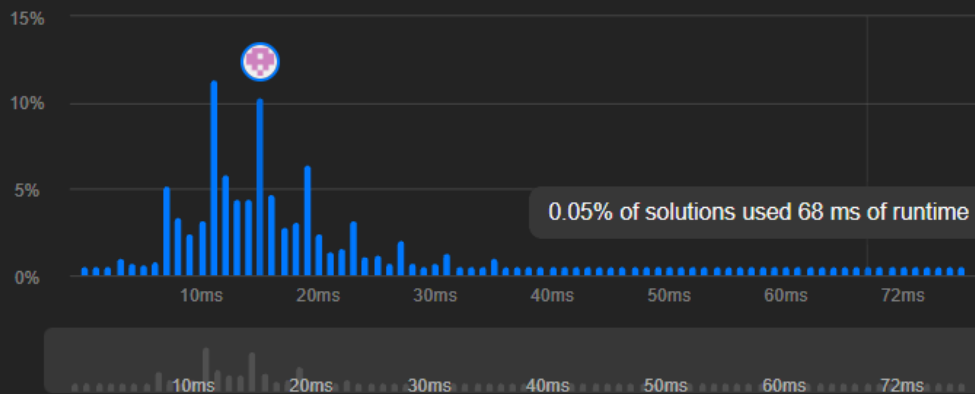
Key Idea: Computing all-pairs shortest paths in a weighted graph to count reachable nodes within a distance threshold.

Runtime

15 ms | Beats 55.86%

Memory

16.64 MB | Beats 67.80%



Lab Program - 8

8a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (5 Marks)

Code:

```
#include <stdio.h>
#define INF 99999
#define MAX 100

void prims(int cost[MAX][MAX], int n) {
    int visited[MAX] = {0};
    int ne = 1;
    int mincost = 0;

    visited[0] = 1;
    printf("\nThe edges of Minimum Cost Spanning Tree are:\n");

    while (ne < n) {
        int min = INF;
        int u = -1, v = -1;

        for (int i = 0; i < n; i++) {
            if (visited[i]) {
                for (int j = 0; j < n; j++) {
                    if (!visited[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }

        if (u != -1 && v != -1) {
            printf("Edge %d: (%d -> %d) cost: %d\n", ne++, u, v, min);
            mincost += min;
            visited[v] = 1;
        }
    }

    printf("\nMinimum Cost: %d\n", mincost);
}
```

```

}

int main() {
    int n, cost[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (99999 for no edge / INF):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j) {
                cost[i][j] = INF;
            }
        }
    }

    prims(cost, n);
    return 0;
}

```

Output:

```

"C:\Users\dell\Desktop\vinila" x + v
Enter number of vertices: 5
Enter adjacency matrix (99999 for no edge / INF):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

The edges of Minimum Cost Spanning Tree are:
Edge 1: (0 -> 1) cost: 2
Edge 2: (1 -> 2) cost: 3
Edge 3: (1 -> 4) cost: 5
Edge 4: (0 -> 3) cost: 6

Minimum Cost: 16

Process returned 0 (0x0)   execution time : 45.379 s
Press any key to continue.

```

8b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm. (5 Marks)

code:

```
#include <stdio.h>
#include <stdlib.h>

#define INF 99999
#define MAX 100

int parent[MAX];

int find(int i) {
    while (parent[i] != i) {
        i = parent[i];
    }
    return i;
}

void unionSets(int i, int j) {
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}

void kruskals(int cost[MAX][MAX], int n) {
    int mincost = 0;
    int edge_count = 0;

    for (int i = 0; i < n; i++) {
        parent[i] = i;
    }

    printf("\nThe edges of Minimum Cost Spanning Tree are:\n");

    while (edge_count < n - 1) {
        int min = INF;
        int u = -1, v = -1;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (find(i) != find(j) && cost[i][j] < min) {
                    min = cost[i][j];
                }
            }
        }

        unionSets(u, v);
        mincost += min;
        edge_count++;
    }
}
```

```

        u = i;
        v = j;
    }
}

if (u != -1 && v != -1) {
    unionSets(u, v);
    printf("Edge %d: (%d -> %d) cost: %d\n", edge_count + 1, u, v, min);
    edge_count++;
    mincost += min;
} else {
    break;
}

printf("\nMinimum Cost: %d\n", mincost);
}

int main() {
    int n, cost[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (99999 for no edge / INF):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j) {
                cost[i][j] = INF;
            }
        }
    }

    kruskals(cost, n);
    return 0;
}

```

output:

```
"C:\Users\dell\Desktop\vinila" X + v
Enter number of vertices: 5
Enter adjacency matrix (99999 for no edge / INF):
0 2 99999 6 99999
2 0 3 8 5
99999 3 0 99999 7
6 8 99999 0 9
99999 5 7 9 0

The edges of Minimum Cost Spanning Tree are:
Edge 1: (0 -> 1) cost: 2
Edge 2: (1 -> 2) cost: 3
Edge 3: (1 -> 4) cost: 5
Edge 4: (0 -> 3) cost: 6

Minimum Cost: 16

Process returned 0 (0x0)   execution time : 54.656 s
Press any key to continue.
|
```

Lab Program - 9

9) Implement Fractional Knapsack using Greedy technique. (5 Marks)

code:

```
#include <stdio.h>

struct Item {
    int id;
    int profit;
    int weight;
    double ratio;
};

void fractionalKnapsack(struct Item items[], int n, int capacity) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (items[j].ratio < items[j + 1].ratio) {
                struct Item temp = items[j];
                items[j] = items[j + 1];
                items[j + 1] = temp;
            }
        }
    }

    double totalProfit = 0.0;
    int currentWeight = 0;

    printf("\nItems selected in the Knapsack:\n");

    for (int i = 0; i < n; i++) {
        if (currentWeight + items[i].weight <= capacity) {
            currentWeight += items[i].weight;
            totalProfit += items[i].profit;
            printf("Item %d: Taken completely (Weight: %d, Profit: %d)\n", items[i].id,
                items[i].weight, items[i].profit);
        } else {
            int remainingCapacity = capacity - currentWeight;
            double fractionProfit = items[i].profit * ((double)remainingCapacity /
                items[i].weight);
            totalProfit += fractionProfit;
        }
    }
}
```

```

        printf("Item %d: Taken fractionally (Weight taken: %d, Profit gained: %.2f)\n",
items[i].id, remainingCapacity, fractionProfit);
        break;
    }
}

printf("\nMaximum Total Profit: %.2f\n", totalProfit);
}

int main() {
    int n;
    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item items[n];
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) {
        items[i].id = i + 1;
        scanf("%d", &items[i].weight);
    }

    printf("Enter profits of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i].profit);
        items[i].ratio = (double)items[i].profit / items[i].weight;
    }

    int capacity;
    printf("Enter capacity of knapsack: ");
    scanf("%d", &capacity);

    fractionalKnapsack(items, n, capacity);

    return 0;
}

```


Output:

```
"C:\Users\dell\Desktop\vinila" × + ▾
Enter number of items: 7
Enter weights of items:
2 3 5 7 1 4 1
Enter profits of items:
10 5 15 7 6 18 3
Enter capacity of knapsack: 12

Items selected in the Knapsack:
Item 5: Taken completely (Weight: 1, Profit: 6)
Item 1: Taken completely (Weight: 2, Profit: 10)
Item 6: Taken completely (Weight: 4, Profit: 18)
Item 3: Taken completely (Weight: 5, Profit: 15)
Item 7: Taken fractionally (Weight taken: 0, Profit gained: 0.00)

Maximum Total Profit: 49.00

Process returned 0 (0x0)   execution time : 24.948 s
Press any key to continue.
```

Lab Program - 10

10) From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int mindis(int dist[], int visited[], int n) {
    int min = INT_MAX, min_index;
    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] <= min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

void dijkstra(int graph[MAX][MAX], int n, int src) {
    int dist[MAX];
    int visited[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = mindis(dist, visited, n);
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
}
```

```

    printf("\nVertex \t Distance\n");
    for (int i = 0; i < n; i++) {
        printf("%d \t\t %d\n", i, dist[i]);
    }
}

int main() {
    int graph[MAX][MAX], n, src;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(graph, n, src);

    return 0;
}

```

output:

```

C:\Users\dell\Desktop\vinila>
Enter number of vertices: 5
Enter adjacency matrix:
0 10 0 30 100
0 0 50 0 0
0 0 0 0 10
0 0 20 0 60
0 0 0 0 0
Enter source vertex: 0

Vertex    Distance
0          0
1          10
2          50
3          30
4          60

Process returned 0 (0x0)   execution time : 49.675 s
Press any key to continue.

```

Lab Program - 11

11) Implement “N-Queens Problem” using Backtracking.

Code:

```
#include <stdio.h>

#define MAX 20

int board[MAX][MAX];
int N;

int isSafe(int row, int col)
{
    int i, j;

    for(i = 0; i < row; i++)
    {
        if(board[i][col])
            return 0;
    }

    for(i = row-1, j = col-1; i >= 0 && j >= 0; i--, j--)
    {
        if(board[i][j])
            return 0;
    }

    for(i = row-1, j = col+1; i >= 0 && j < N; i--, j++)
    {
        if(board[i][j])
            return 0;
    }

    return 1;
}

void printBoard()
{
    for(int i = 0; i < N; i++)
    {
        for(int j = 0; j < N; j++)
```

```

        {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

```

```

void solveNQueens(int row)
{
    if(row == N)
    {
        printBoard();
        return;
    }

    for(int col = 0; col < N; col++)
    {
        if(isSafe(row, col))
        {
            board[row][col] = 1;

            solveNQueens(row + 1);

            board[row][col] = 0;
        }
    }
}

```

```

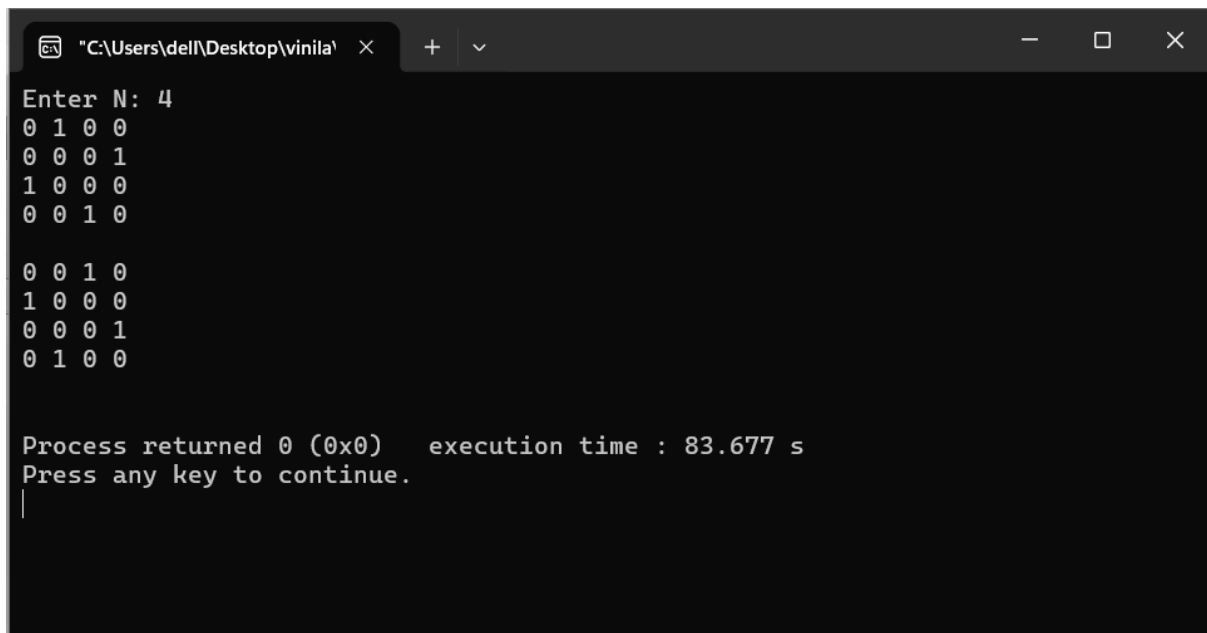
int main()
{
    printf("Enter N: ");
    scanf("%d", &N);

    solveNQueens(0);

    return 0;
}

```

Output:



```
"C:\Users\dell\Desktop\vinila" × + ▾  
Enter N: 4  
0 1 0 0  
0 0 0 1  
1 0 0 0  
0 0 1 0  
  
0 0 1 0  
1 0 0 0  
0 0 0 1  
0 1 0 0  
  
Process returned 0 (0x0)   execution time : 83.677 s  
Press any key to continue.  
|
```